EnviroWatch – AQI & Weather Monitoring Dashboard

A PROJECT REPORT

Mini Project (01CE0609)

Submitted by

Rajnish Sinh
92410103096

Kishan Vachhani
92410103097

BACHELOR OF TECHNOLOGY
in
Computer Engineering

Faculty of Engineering & Technology
Marwadi University, Rajkot

April, 2026

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled EnviroWatch – AQI & Weather Monitoring Dashboard has been carried out by Rajnish Sinh (92410103096), Kishan Vachhani (92410103097) under my guidance in partial fulfillment for the degree of Bachelor of Technology in Computer Engineering, 6th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Viraj Daxini	Prof. (Dr.) Krunal Vaghela
Assistant Professor	Professor & Associate Dean
Department of Computer Engineering	Department of Computer Engineering

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

DECLARATION

We hereby declare that the **Mini Project (01CE0609)** report submitted along with the Project entitled EnviroWatch – AQI & Weather Monitoring Dashboard submitted in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering to Marwadi University, Rajkot, is a bonafide record of original project work carried out by me / us at Marwadi University under the supervision of **Prof. Viraj Daxini** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

	Name of the Student	Sign of Student
1	_____	_____
2	_____	_____

Acknowledgement

We would like to express our sincere gratitude to our respected guide Prof. Viraj Daxini, Department of Computer Engineering, Marwadi University, Rajkot, for his valuable guidance, continuous support, and motivation throughout the development of this mini project EnviroWatch – AQI & Weather Monitoring Dashboard. His technical knowledge and suggestions helped us to successfully complete this project.

We are also thankful to the Department of Computer Engineering, Faculty of Engineering & Technology, Marwadi University for providing the necessary infrastructure and development environment required for completing this project.

We would like to extend our thanks to all the faculty members who directly or indirectly helped us by providing their valuable suggestions and encouragement during the project development.

Finally, we would like to thank our family members for their constant encouragement, moral support, and motivation which helped us to complete this project successfully.

Abstract

*The **EnviroWatch – AQI & Weather Monitoring Dashboard** is a web-based solution designed to address this need by providing live environmental insights through an interactive and easy-to-use interface. Built using the ASP.NET Core MVC framework, the system integrates external APIs to fetch real-time AQI and weather data, which is then processed and displayed using visual charts and structured formats for better understanding.*

The application includes several useful features such as secure user authentication, real-time AQI and weather tracking, alert notifications for hazardous conditions, and personalized health recommendations based on pollution levels. It also offers district-wise environmental analysis and allows users to export reports for further use. The alert system ensures users stay informed, while the recommendation module promotes preventive actions to reduce health risks.

Overall, the project aims to create a centralized platform that not only increases environmental awareness but also encourages safer lifestyle choices, while showcasing the practical use of modern web technologies in solving real-world problems.

List of Figures

Fig 1.1 Gantt Chart 3
Fig 3.1 System Architecture 6
Fig 3.2 Physical ER Diagram 7
Fig 3.3 DFD Diagram Level 0 8
Fig 3.4 DFD Diagram Level 1 9
Fig 3.5 Use Case Diagram 10
Fig 3.6 Class Diagram 11
Fig 3.7 Sequence Diagram 12
Fig 3.8 Activity Diagram..... 13
Fig 3.9 Main Dashboard 14
Fig 3.10 Login Page 15
Fig 3.11 Alert Subscription 15
Fig 3.12 PDF Report 16

List of Tables

Table 4.6 Test Cases.....19

Abbreviations

API	Application Programming Interface
AQI	Air Quality Index
ASP.NET Core	Active Server Pages Network Enabled Technologies Core
CO	Carbon Monoxide
CSS	Cascading Style Sheets
EF Core	Entity Framework Core
HTML	Hypertext Markup Language
HTTP / HTTPS	Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
MVC	Model-View-Controller
NO2	Nitrogen Dioxide
OAuth	Open Authorization
PM2.5 / PM10	Particulate Matter 2.5 / Particulate Matter 10 (micrometers or less in diameter)
PWA	Progressive Web App
SO2	Sulfur Dioxide
SQL	Structured Query Language
UI / UX	User Interface / User Experience

Table of Contents

Acknowledgement ..... **i**

Abstract ..... **ii**

List of Figures ..... **iii**

List of Tables ..... **iv**

List of Abbreviations ..... **v**

Table of Contents ..... **vi**

1 Introduction to Project 1

1.1 Project Summary 1

1.2 Purpose 1

1.3 Objective 1

1.4 Scope 2

1.5 Technology and Literature Review 2

1.6 Project Planning and Scheduling 2

1.6.1 Project Development Approach and Justification 2

1.6.2 Project Effort and Time, Cost Estimation 3

1.6.3 Roles and Responsibilities 3

1.6.4 Group Dependencies 3

1.7 Project Scheduling (Gantt Chart/PERT/Network Chart) 4

2 System Analysis 5

2.1 Study of Current System 5

2.2 Problem and Weaknesses of Current System 5

2.3 Requirements of New System 5

2.4 System Feasibility..... 6

2.4.1 Does the system contribute to the overall objectives of the organization? 6

2.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints. 6

2.4.3 Can the system be integrated with other systems which are already in place? 6

2.5 Activity / Process in New System / Proposed System 6
2.6 Features of New System / Proposed System 6
2.7 List Main Modules / Components 7
2.8 Selection of Hardware / Software / Methodology 7
3 System Design 8
3.1 System Design & Methodology 8
3.2 ER Diagram 8
3.3 Input / Output and Interface Design 16
3.3.1 Samples of Forms, Reports and Interface 16
3.3.2 Access Control / Mechanism / Security 20
4 Implementation & Testing 21
4.1 Implementation Platform / Environment 21
4.2 Process / Program / Technology / Modules Specification(s) 21
4.3 Finding / Results / Outcomes 21
4.4 Result Analysis / Comparison / Deliberations 21
4.5 Testing Plan / Strategy..... 21
4.6 Test Results and Analysis 22
4.6.1 Test Cases 22
5.0 Conclusion and Future Enhancements 23
5.1 Overall Analysis of Project Viabilities 23
5.2 Problem Encountered and Possible Solutions 23
5.3 Summary of Project work 23
5.4 Limitations 23
5.5 Future Enhancement 24
5.6 Project Outcomes and discussion 24
References **25**

CHAPTER 1

INTRODUCTION TO PROJECT

1.1 Project Summary :

EnviroWatch is a comprehensive web-based dashboard designed to provide real-time Air Quality Index (AQI) and weather data. Developed using modern web development frameworks (ASP.NET Core 8.0), the platform visually represents environmental data using interactive maps, allowing users to track real-time conditions for various geographical areas.

1.2 Purpose :

The primary purpose of EnviroWatch is to increase environmental awareness by providing users with easily accessible, accurate, and localized data regarding air quality and weather. By consolidating data that is often scattered across multiple platforms into a single, intuitive interface, EnviroWatch aims to help people make informed decisions regarding their daily activities.

1.3 Objective :

- To fetch and display real-time AQI and weather metrics using responsive external REST APIs.
- To present data visually through dynamic geographical maps and interactive charts.
- To allow users to export environmental data reports in PDF formats.
- To implement a secure authentication system, including Third-Party integrations (Google, GitHub).
- To provide subscription-based premium features for power users.

1.4 Scope :

What it can do:

- Display real-time temperature, humidity, wind conditions, and AQI status.
- Visualize data on an interactive map tailored to specific districts/regions.
- Provide a premium subscription tier with advanced environmental data monitoring functionality.
- Provide data export capabilities using QuestPDF.
- Manage user authentication using ASP.NET Core Identity.

What it can't do:

- It does not predict or forecast future AQI conditions beyond what external APIs provide.
- It does not natively collect field data via edge hardware/sensors; it strictly acts as a consumer of existing centralized environmental APIs.
- It cannot manipulate or change the source data directly.

1.5 Technology and Literature Review :

- Backend: ASP.NET Core 8.0 (C#)
- Database: Microsoft SQL Server with Entity Framework Core
- Frontend UI: HTML5, CSS3, JavaScript (Leaflet.js for mapping, custom JS for interactivity), Bootstrap
- Document Generation: QuestPDF
- Authentication: ASP.NET Identity, OAuth (Google, GitHub)

1.6 Project Planning and Scheduling :

1.6.1 Project Development Approach and Justification :

The Agile methodology was chosen for this project. Agile allows for iterative development, dividing the project into small, manageable sprints. Because UI design and API integration often require rapid adjustments based on data availability and

user feedback, Agile enables the team to adapt to changes fluidly without derailing the entire schedule.

1.6.2 Project Effort and Time, Cost Estimation :

- Effort Estimation: Divided into Frontend UI Design, Backend API Integration, Database structuring, and User Authentication configuration.
- Time Estimation: Completed over the course of the semester, broken down into requirements gathering (weeks 1-2), design (weeks 3-4), implementation and API integration (weeks 5-8), and testing/deployment (weeks 9-10).
- Cost Estimation: Minimal financial cost, utilizing open-source libraries (e.g., Leaflet), free tiers of external APIs, and developer tools like Visual Studio Community Edition.

1.6.3 Roles and Responsibilities :

- Project Manager / Backend Developer: Responsible for the ASP.NET Core architecture, database schemas, and external API service configuration.
- Frontend Developer: Tasked with building the user interface, incorporating map libraries, and ensuring mobile responsiveness.
- QA & Documentation Lead: Responsible for writing and executing test cases, debugging, and compiling this project report.

1.6.4 Group Dependencies :

The frontend components heavily depend on the completion of backend REST service wrappers (like AQIService.cs). Data mapping and visualization tasks are blocked until valid, standardized JSON data is successfully routed from the backend.

1.7 Project Scheduling :

The project was scheduled based on the following milestones:

Phase 1: Requirement analysis and API selection.

Phase 2: Setup of ASP.NET Core project and database configuration via Entity Framework.

Phase 3: Development of core services (AQI Service, Weather Service).

Phase 4: Frontend implementation, integration of interactive map (Leaflet.js).

Phase 5: Implementation of Identity authentication and PDF Export features.

Phase 6: Testing, debugging, and documentation preparation.

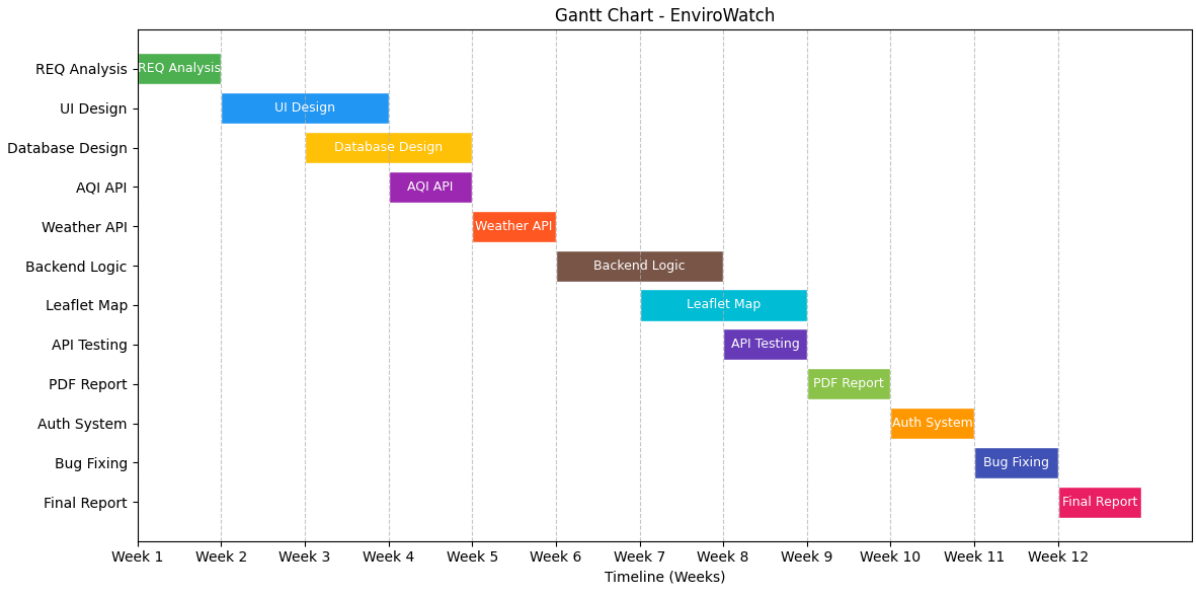


Figure 1.1 Gantt Chart

CHAPTER 2

SYSTEM ANALYSIS

2.1 Study of Current System :

Currently, users looking for environmental data often have to visit weather-specific websites for temperature/forecasts and scientific/government sites for air quality indexes. Many of these sites are heavily cluttered with ads, have poor UI localization, or fail to combine AQI and basic weather stats effectively in a single dashboard view.

2.2 Problem and Weaknesses of Current System :

- Fragmented Information: Weather arrays and AQI metrics reside on entirely different websites.
- Poor Visualizations: Existing portals often present data in static tables, lacking dynamic maps.
- No Exporting: Lack of functionality to download environmental reports in organized PDF formats.
- Poor UX: Many existing platforms do not utilize modern "SaaS-style" dashboard aesthetics.

2.3 Requirements of New System :

- A unified interface combining weather metrics and air quality.
- Interactive geographic mapping pointing to user-selected locations.
- A secure user authentication model.
- Premium subscription management for power users.
- PDF report generation for recorded data.

2.4 System Feasibility :

2.4.1 Does the system contribute to the overall objectives of the organization?

Yes, it creates a streamlined, highly functional utility that encourages environmental awareness, combining technical learning objectives with a practical software engineering product.

2.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints?

Yes, using .NET 8.0, free external API tiers (e.g., OpenWeatherMap, AQICN), and standard web technologies guarantees that there are no blocking financial or technological constraints.

2.4.3 Can the system be integrated with other systems which are already in place?

Through OAuth packages, the system readily integrates with standard identity providers (Google, GitHub) existing in the global ecosystem.

2.5 Activity / Process in New System / Proposed System :

1. User lands on the dashboard and is presented with localized environmental data.
2. User navigates the interactive map, selecting different regions/districts via configured JS data structures.
3. The system makes asynchronous calls to backend services (AQIService).
4. Data is parsed and returned to the UI dynamically.
5. Users can log in using OAuth to access premium subscription layers and export PDF reports using the configured IExportService.

2.6 Features of New System / Proposed System :

- Unified Dashboard: Displays AQI, temperature, humidity, and wind.
- Interactive Maps: Powered by JS map integration for visual geographical scanning.
- Authentication & Profiles: Secure user registration, login, and customized settings.
- PDF Report Engine: Exporting map/AQI context into styled documents.
- Subscription Module: Handling access tiers for standard vs. premium users.

2.7 List Main Modules / Components :

1. Authentication Module: Handles login, registers, Google/GitHub OAuth.
2. Dashboard/Map Module: The graphical interface (Index.cshtml, map.js).
3. Data Fetching Service: C# REST wrappers to fetch live environmental data (AQIService.cs).
4. Export/Reporting Module: Generates PDF documents based on given conditions (IExportService.cs).
5. Subscription Tier Module: Manages user access levels (SubscriptionsController.cs).

2.8 Selection of Hardware / Software / Methodologies :

- ASP.NET Core 8: Chosen for its extremely high performance, robust dependency injection, and secure Identity framework.
- Entity Framework Core: Selected to avoid raw SQL generation, preventing SQL injection and streamlining database modeling.
- QuestPDF: Chosen over alternatives due to its modern, layout-driven, and highly performant C#-based PDF generation approach.

CHAPTER 3

SYSTEM DESIGN

3.1 System Design & Methodology :

The project strictly follows the Model-View-Controller (MVC) architectural pattern.

- Models: Represent the database schema (Users, Subscriptions) and data payloads (Weather JSON responses).
- Views: Razor pages (.cshtml) paired with CSS and Javascript serving dynamic HTML.
- Controllers: Handle user routing and HTTP requests, acting as the intermediary between the backend Services and the frontend Views.

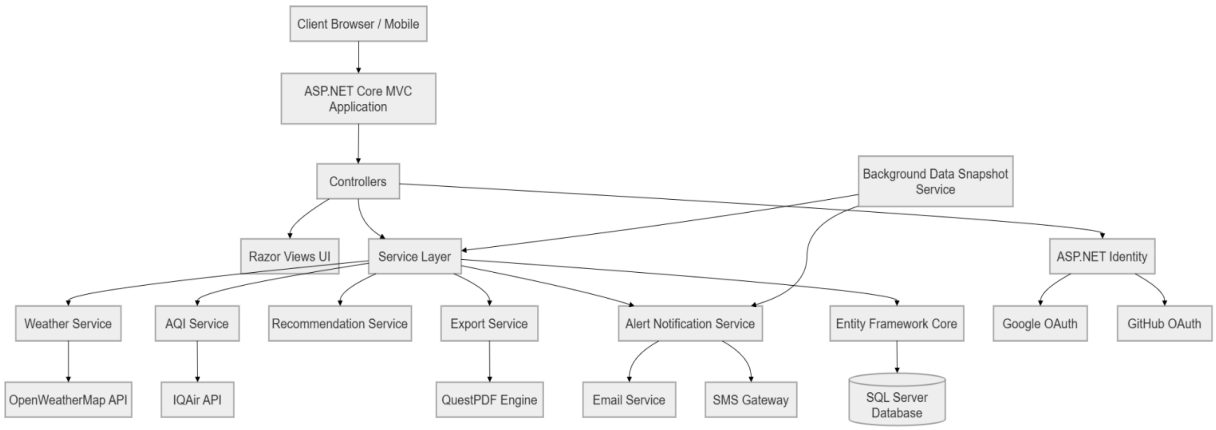


Figure 3.1 System Architecture

3.2 Database Design / ER Diagram / DFD :

Brief Database Structure:

- AspNetUsers: Stores user credentials, emails, and Identity details.
- Subscriptions: Links a UserId to a subscription tier (Free, Premium) tracking expiry and feature limits.
- Data Flow: User UI Request -> Controller -> AQI Service -> External API -> Return JSON -> Controller formats -> View parses data -> Map updates.

3.2.2 ER Diagram

Physical ER Diagram

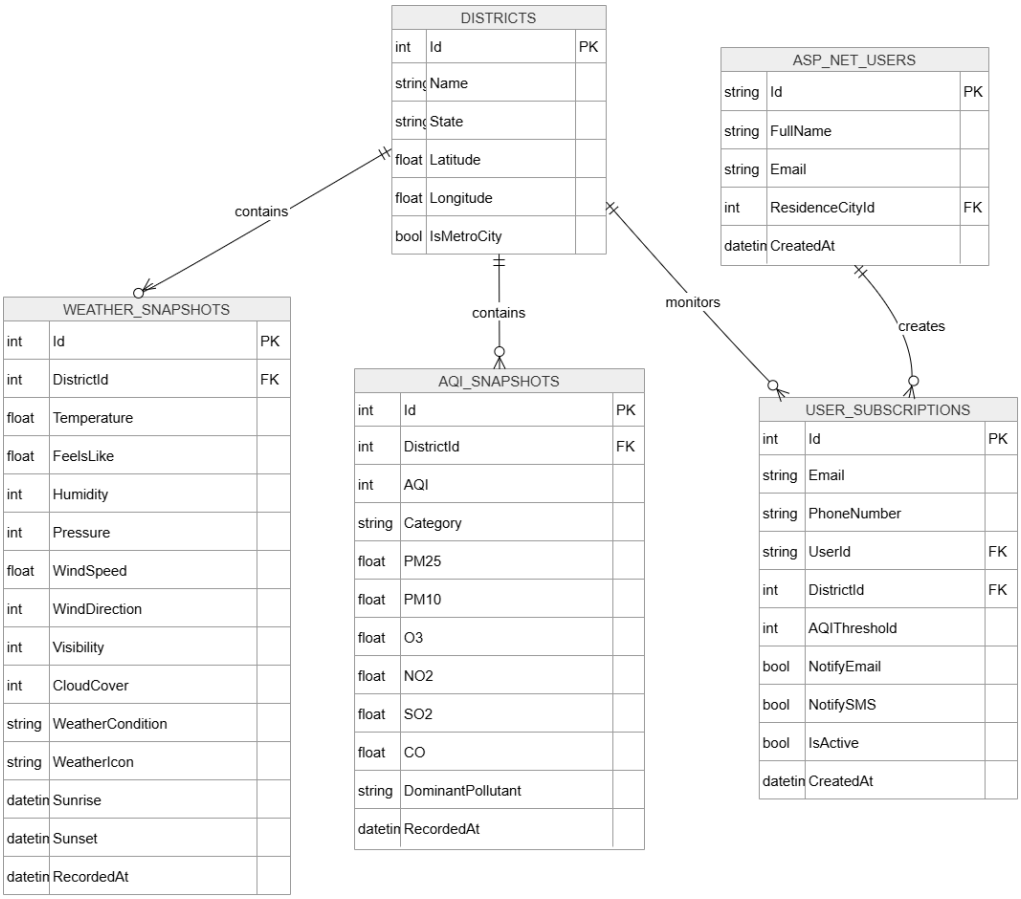


Figure 3.2 Phsical ER Diagram

3.2.3 Data Flow Diagram

Level 0 – Context Diagram

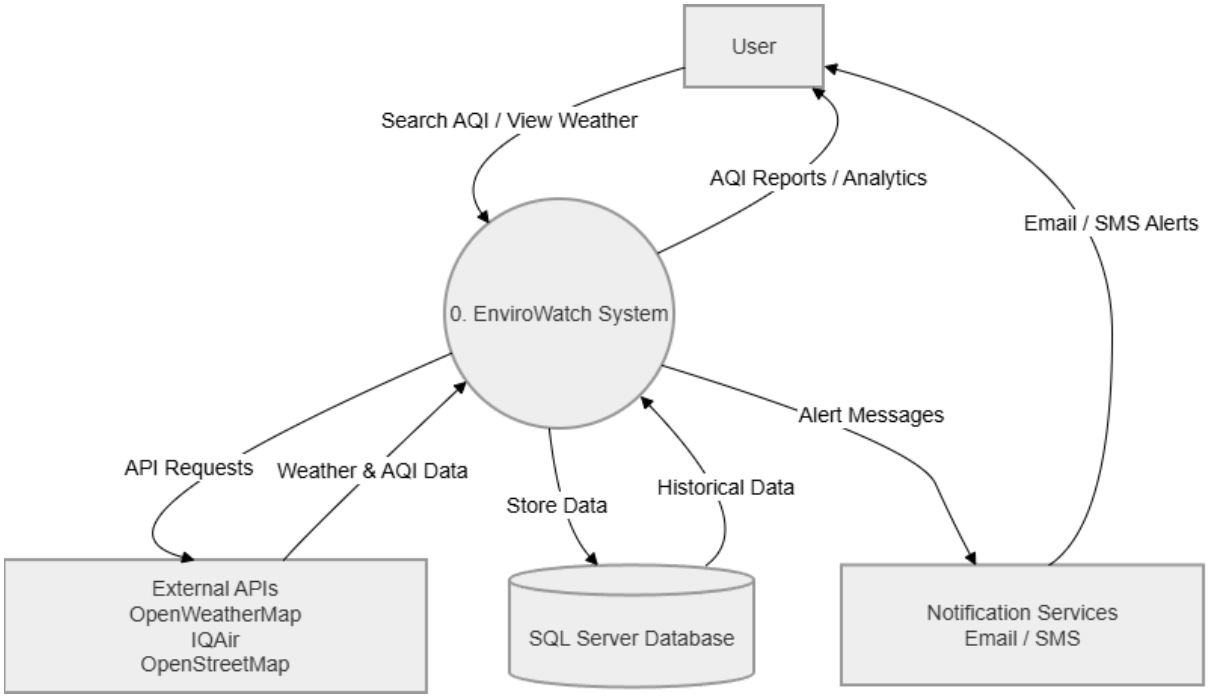


Figure 3.3 DFD Diagram Level 0

Level 1 – Detailed Diagram

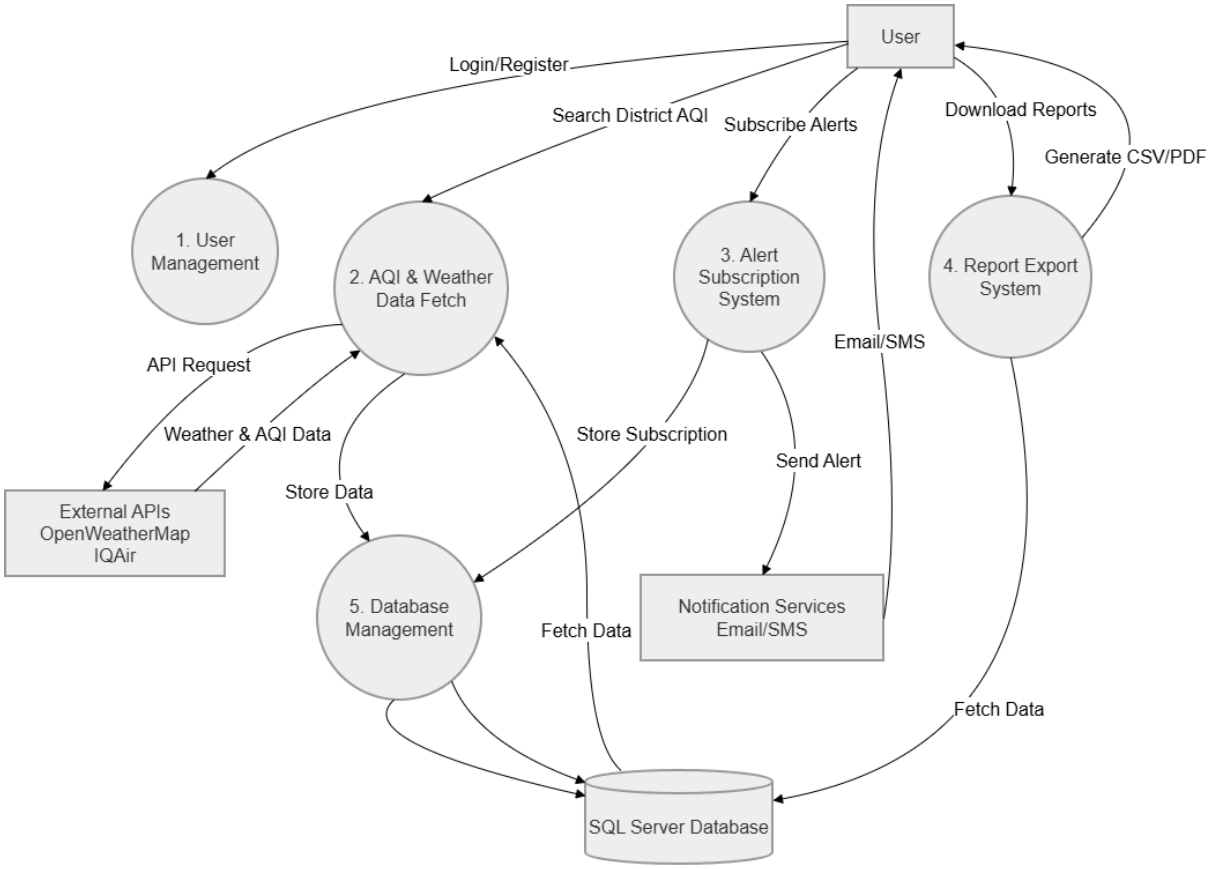


Figure 3.4 DFD Diagram Level 1

3.2.4 UML – Use case Diagram

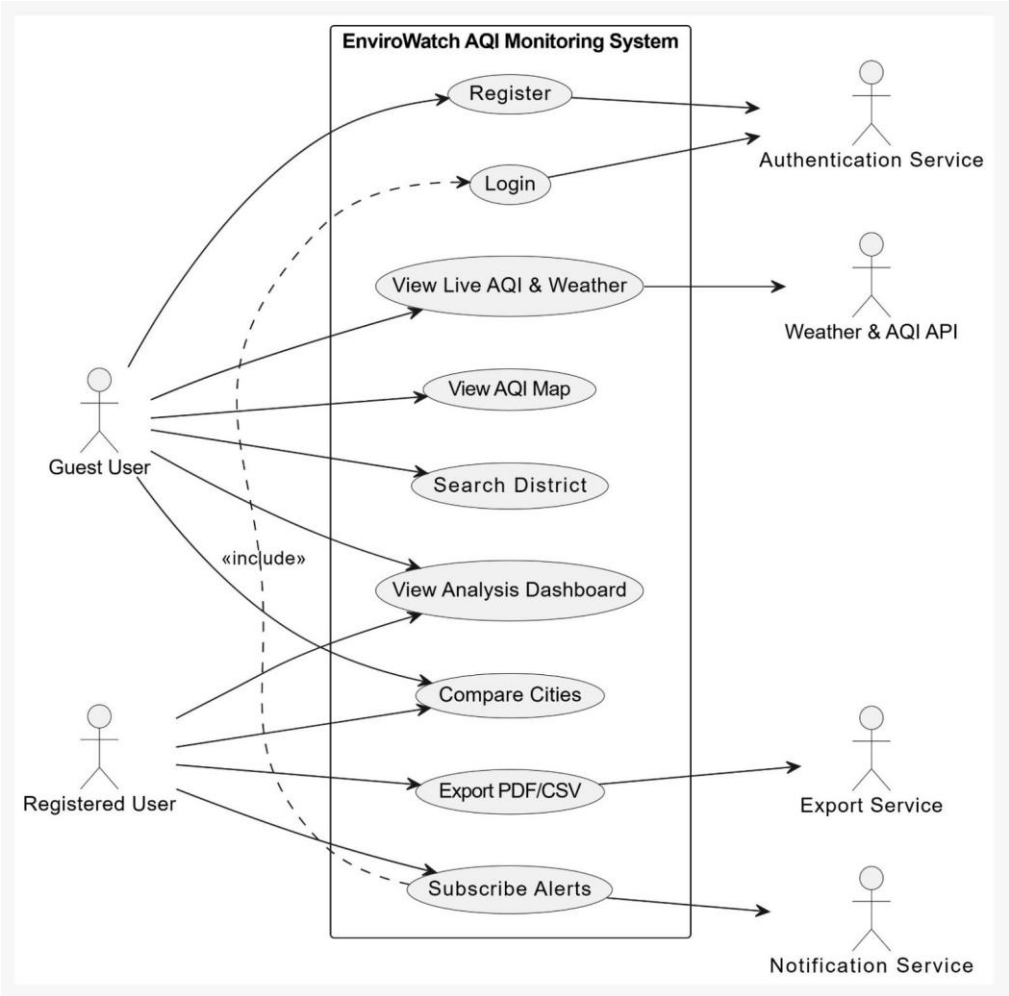


Figure 3.5 Use Case Diagram

3.2.5 UML – Class Diagram

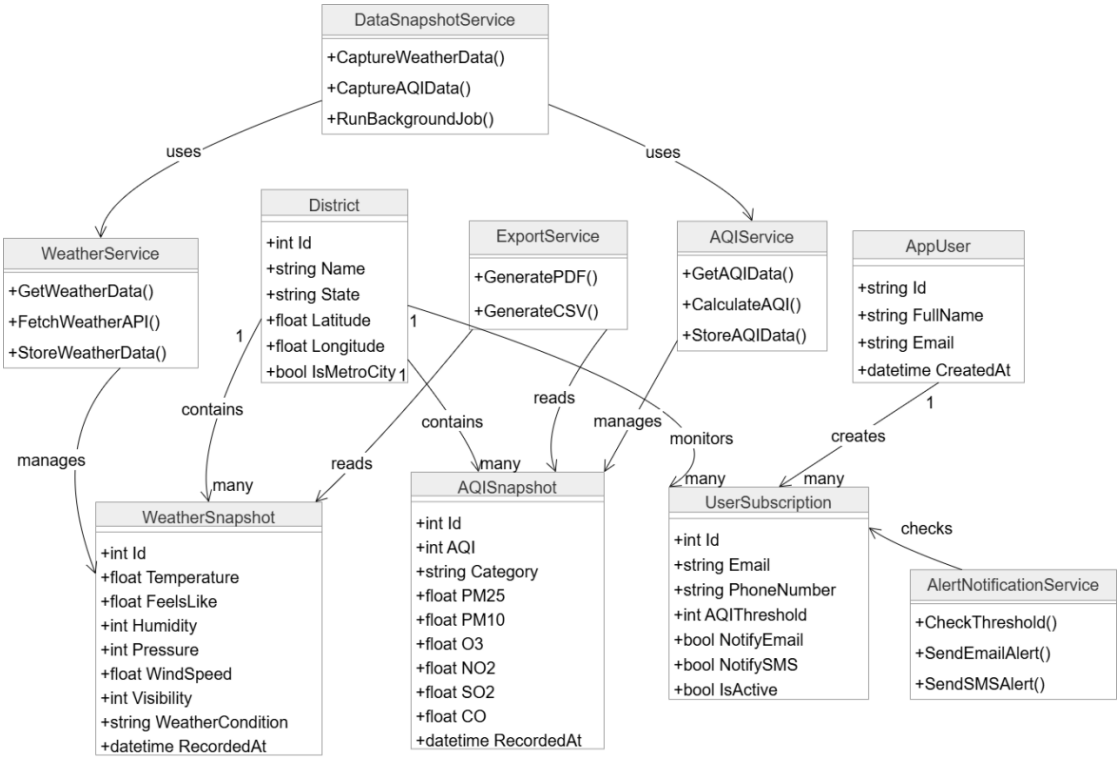


Figure 3.6 Class Diagram

3.2.6 Sequence Diagram

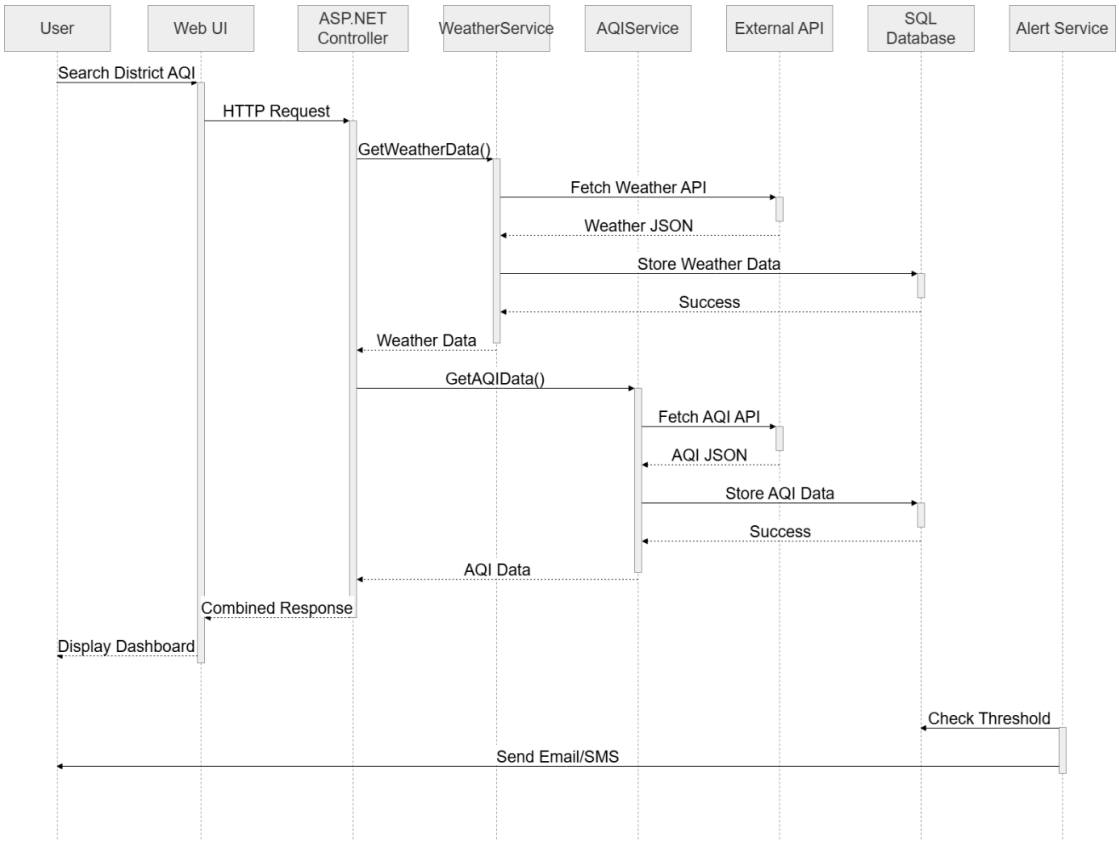


Figure 3.7 Sequence Diagram

3.2.7 Activity Diagram

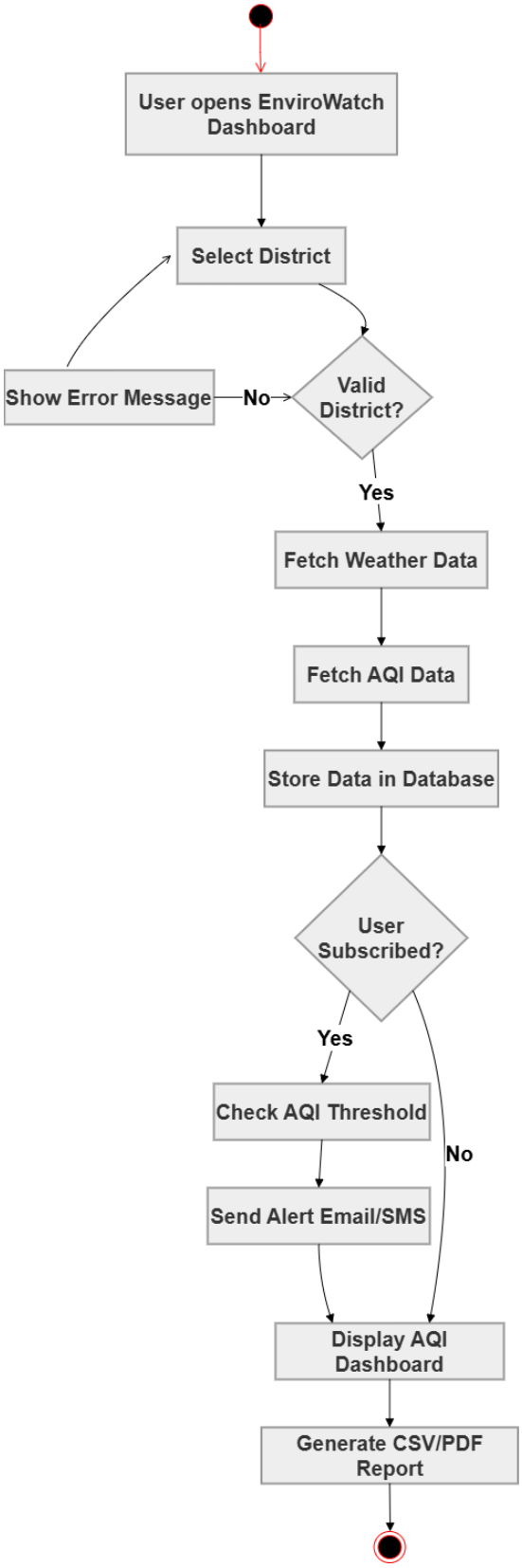


Figure 3.8 Activity Diagram

3.3 Input / Output and Interface Design :

3.3.1 Samples of Forms, Reports and Interface :

1. **Main Dashboard Interface:** The primary user interface features a wide, interactive geographical map powered by Leaflet.js. Users can click on designated geographical boundaries (e.g., specific districts). Accompanying the map are highly readable, dynamic "Data Cards." When a region is selected on the map, these cards instantly update via asynchronous backend calls to display the localized Air Quality Index (AQI), temperature, wind speed, and humidity in a clean, modern grid layout.

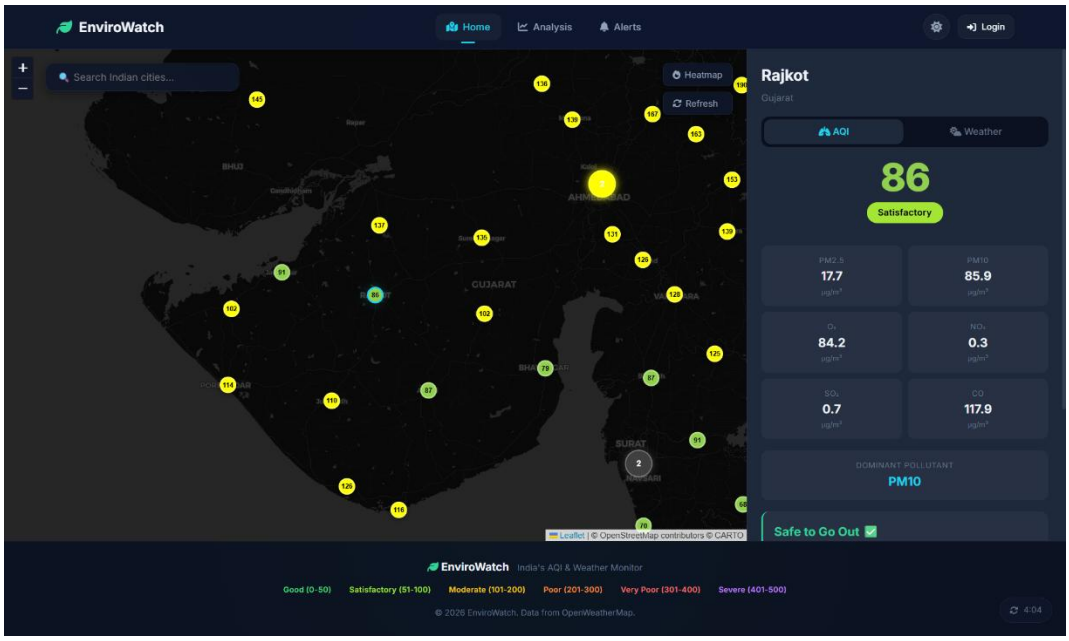


Figure 3.9 Main Dashboard

2. **Authentication Forms (Login / Registration):** The application includes secure, user-friendly login and registration forms built over ASP.NET Core Identity. The forms prompt users for standard email and password combinations while enforcing security constraints (e.g., minimum password complexity). Additionally, the interface features prominent "Log in with Google" and "Log in with GitHub" buttons, allowing users a seamless OAuth integration pathway that bypasses the need for manual password entry.

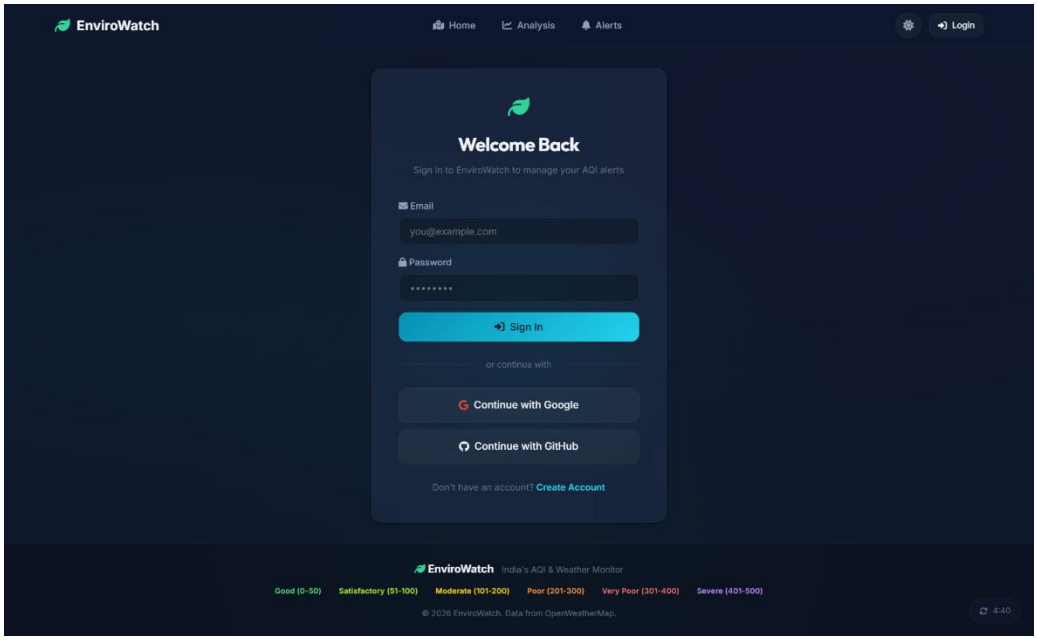


Figure.3.10 Login Page

3. **Subscription Management Interface:** A dedicated view lists the available account tiers (Free vs. Premium). The interface is designed as a comparison table or side-by-side card layout, clearly defining the benefits of the premium tier (e.g., unlimited PDF exports, specialized notifications). It includes clear input triggers for users to upgrade or downgrade their account status.

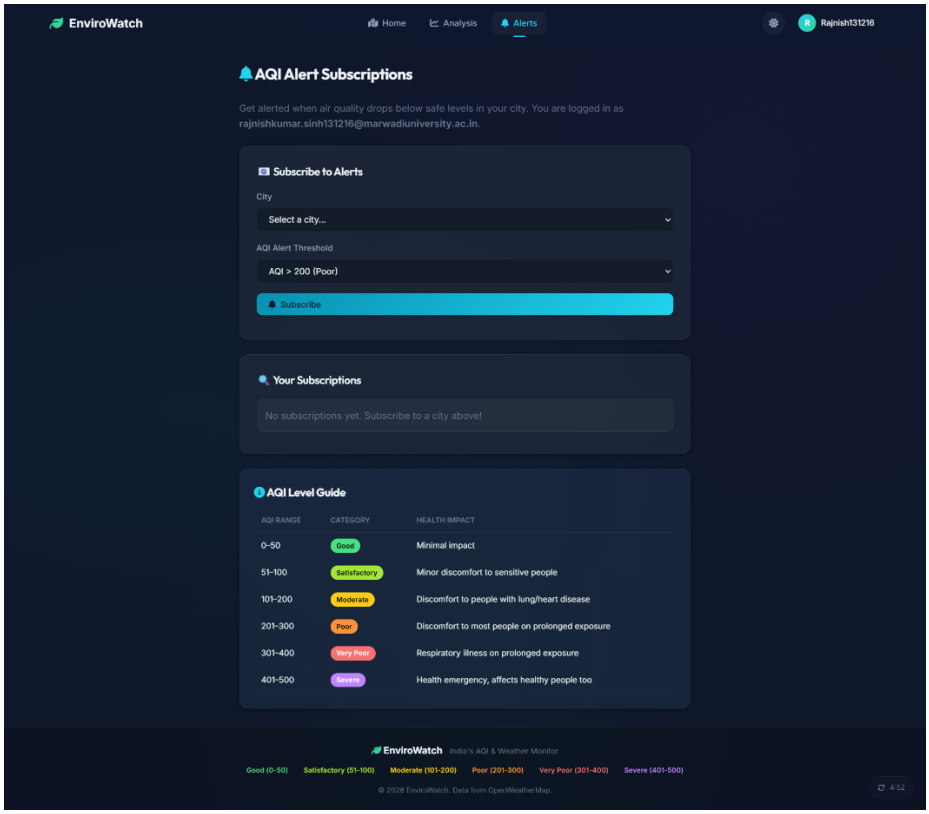


Figure 3.11 Alert Subscription

4. Generated PDF Report: Instead of a simple web printout, the project includes structured PDF reports dynamically constructed via the QuestPDF library. The output is a highly professional, stylized document featuring:
    - A branded "EnviroWatch" header with a generated timestamp.
    - A localized summary of the queried AQI and weather parameters.
    - Tabular breakdowns of the environmental data, perfectly scaled for A4 printing and archival purposes.

EnviroWatch — Air Quality & Weather Report

Rajkot | Last 30 Days | Generated: 09 Apr 2026 14:04

Current Summary

AQI	86 (Satisfactory)	**Dominant**	PM10
Temperature	32.75°C (Feels 31.69°C)	**Humidity**	30%

AQI History

Date	AQI	Category	PM2.5	PM10	Dominant
09-04 08:28	86	Satisfactory	17.7	85.9	PM10
09-04 07:15	88	Satisfactory	18.2	87.6	PM10
09-04 06:33	88	Satisfactory	18.2	87.6	PM10
09-04 06:18	88	Satisfactory	18.2	88.2	PM10
09-04 06:02	88	Satisfactory	18.2	88.2	PM10
09-04 05:36	88	Satisfactory	18.2	88.2	PM10
09-04 05:00	86	Satisfactory	17.6	85.8	PM10
09-04 05:00	86	Satisfactory	17.6	85.8	PM10
09-04 04:00	82	Satisfactory	17.2	82.3	PM10
09-04 04:00	82	Satisfactory	17.2	82.3	PM10
09-04 03:00	75	Satisfactory	17.6	75.4	PM10
09-04 03:00	75	Satisfactory	17.6	75.4	PM10
09-04 02:00	71	Satisfactory	17.3	70.9	PM10
09-04 02:00	71	Satisfactory	17.3	70.9	PM10
09-04 01:00	68	Satisfactory	16.8	67.9	PM10
09-04 01:00	68	Satisfactory	16.8	67.9	PM10
09-04 00:00	67	Satisfactory	16.7	67.0	PM10
09-04 00:00	67	Satisfactory	16.7	67.0	PM10
08-04 23:00	67	Satisfactory	16.8	67.5	PM10
08-04 23:00	67	Satisfactory	16.8	67.5	PM10
08-04 22:00	69	Satisfactory	17.1	69.0	PM10
08-04 22:00	69	Satisfactory	17.1	69.0	PM10
08-04 21:00	71	Satisfactory	17.6	71.0	PM10
08-04 21:00	71	Satisfactory	17.6	71.0	PM10
08-04 20:00	73	Satisfactory	18.2	72.8	PM10
08-04 20:00	73	Satisfactory	18.2	72.8	PM10
08-04 19:00	74	Satisfactory	18.5	73.9	PM10
08-04 19:00	74	Satisfactory	18.5	73.9	PM10
08-04 18:00	74	Satisfactory	18.7	74.2	PM10
08-04 18:00	74	Satisfactory	18.7	74.2	PM10
08-04 17:00	73	Satisfactory	18.3	72.8	PM10
08-04 17:00	73	Satisfactory	18.3	72.8	PM10
08-04 16:00	71	Satisfactory	17.6	71.0	PM10
08-04 16:00	71	Satisfactory	17.6	71.0	PM10
08-04 15:00	70	Satisfactory	16.8	68.7	O3
08-04 15:00	70	Satisfactory	16.8	68.7	O3
08-04 14:00	68	Satisfactory	15.5	64.9	O3
08-04 14:00	68	Satisfactory	15.5	64.9	O3

EnviroWatch © 2026 | Page 1 of 65

Figure 3.12 PDF Report

3.3.2 Access Control / Mechanism / Security :

- Identity Framework: Uses secure password hashing mechanics by default.
- OAuth2: Relies on tokens provided securely by third parties (Google/GitHub) preventing raw password storage if desired.
- Authorization Tags: Controllers (like SubscriptionsController) utilize the [Authorize] attributes to ensure anonymous users cannot access premium endpoints.

CHAPTER 4

IMPLEMENTATION & TESTING

4.1 Implementation Platform / Environment :

- IDE: Visual Studio / Visual Studio Code
- Runtime: .NET 8.0 SDK
- Database Server: Microsoft SQL Server LocalDB
- Browser: Google Chrome (Primary), Firefox/Edge for cross-compatibility checks.

4.2 Process / Program / Technology / Modules Specification(s) :

- AQIService.cs: Uses HttpClient specifically configured with IHttpClientFactory for efficient, thread-safe external REST API polling.
- map.js: Uses modern ES6+ Javascript interacting directly with Leaflet API or similar geo-renderers.
- IExportService.cs: Abstracted interface using QuestPDF to compose multi-page reports.

4.3 Finding / Results / Outcomes :

The developed application correctly parses highly-nested JSON data from 3rd party providers, maps it consistently to internal C# objects, and handles asynchronous events on the frontend gracefully without freezing the UI.

4.4 Result Analysis / Comparison / Deliberations :

Compared to using static console applications or separated mobile apps, combining mapping and external data into a unified, responsive web template provided a significantly superior User Experience.

4.5 Testing Plan / Strategy :

The platform utilized Manual Testing alongside targeted logical debugging. Testing strategies included:

- Unit Testing (Informal): Verifying specific service methods returned expected null values when given bad API keys.

- Integration Testing: Ensuring the database correctly registered third-party OAuth logins.
- UI/Responsive Testing: Expanding and collapsing the browser window to verify flexbox and grid behaviors at mobile widths.

4.6 Test Results and Analysis :

4.6.1 Test Cases :

Table 4.6 Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
TC-01	Enter invalid external API location query	Service handles 404 cleanly, returns friendly error msg	Clean error message shown	Pass
TC-02	Unauthenticated user visits Subscription page	System redirects to Login Page	Redirected to /Identity/Account/ Login	Pass
TC-03	Export PDF with data loaded on dashboard	PDF generated and triggers browser download	PDF file downloaded via controller	Pass
TC-04	Register an account using Google Auth	Account created in DB without local password requirement	Account registered	Pass
TC-05	Click district on customized map view	AQI/Weather cards dynamically update info	Cards updated dynamically	Pass

CHAPTER 5

CONCLUSION & FUTURE ENHANCEMENT

5.1 Overall Analysis of Project Viabilities :

EnviroWatch is a highly viable web application that delivers strong commercial presentation alongside practical environmental utility. Its robust backend architecture (.NET 8 MVC) gives it a strong foundation for scaling up.

5.2 Problem Encountered and Possible Solutions :

- Problem: API Rate limits restricting the amount of live data that can be fetched mapped simultaneously.
  - Solution: Implemented backend caching to store AQI results for 10-15 minutes, heavily reducing outbound requests to the external API providers.
- Problem: Mapping performance on slower client devices when loading massive geo-json files.
  - Solution: Minimized indian_districts.json and utilized optimized coordinate clustering.

5.3 Summary of Project work :

In summary, the EnviroWatch project successfully bridged external environmental REST APIs with a visually striking, map-driven frontend interfaces. The database safely manages users and subscriptions, while specialized services handle complex operations like PDF exporting seamlessly.

5.4 Limitations :

- The system is entirely reliant on the uptime of third-party API providers (OpenWeather, AQICN). If their servers drop, the application loses its core utility.
- Only specific geographical zones outlined in the configured JSON are fully supported with localized boundary data.

5.5 Future Enhancement :

- Push Notifications: Allow users to set up alerts to their email/phone when the AQI in their subscribed region drops below a certain threshold.
- Historical Data Graphing: Instead of only showing real-time data, fetch historical metrics to draw line charts showing trends over the last 30 days.
- Mobile App Iteration: Package the current dashboard using progressive application frameworks or port to Flutter for native mobile distribution.

5.6 Project Outcomes and discussion :

The final outcome of this project is a fully functional, cloud-ready ASP.NET Core MVC web application. It securely integrates modern authentication, interactive web mapping, service-oriented data fetching, and PDF generation inside an aesthetically pleasing and highly responsive design system, successfully meeting all fundamental requirements set out in the initial project scope.

References

[1] Microsoft, ".NET and ASP.NET Core Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/dotnet/.

[2] Microsoft, "Entity Framework Core Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/ef/core/.

[3] OpenWeather, "OpenWeatherMap API Reference," OpenWeather, [Online]. Available: https://openweathermap.org/api.

[4] IQAir, "IQAir AirVisual Enterprise API," IQAir, [Online]. Available: https://www.iqair.com/enterprise/api.

[5] Microsoft, "SQL Server Technical Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/sql/sql-server/.

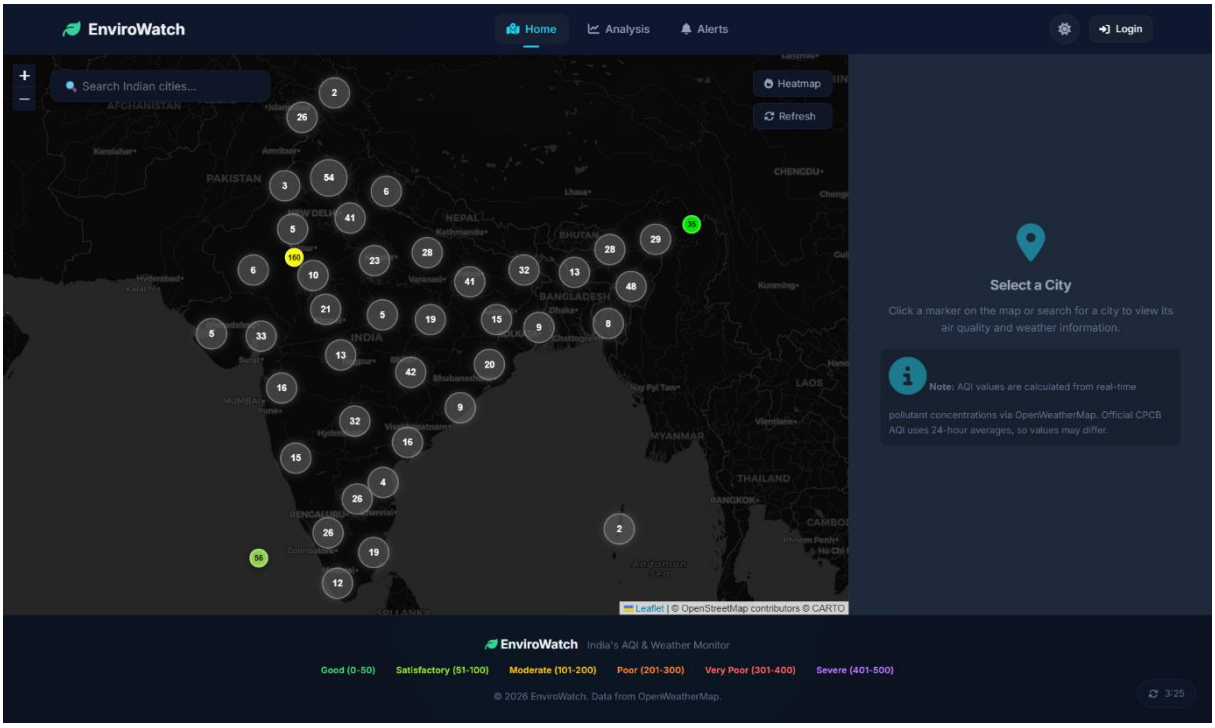
[6] Mozilla Developer Network, "Progressive Web Apps (PWA)," MDN Web Docs, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps.

[7] Google, "Google Identity Services and OAuth 2.0," Google Developers, [Online]. Available: https://developers.google.com/identity.

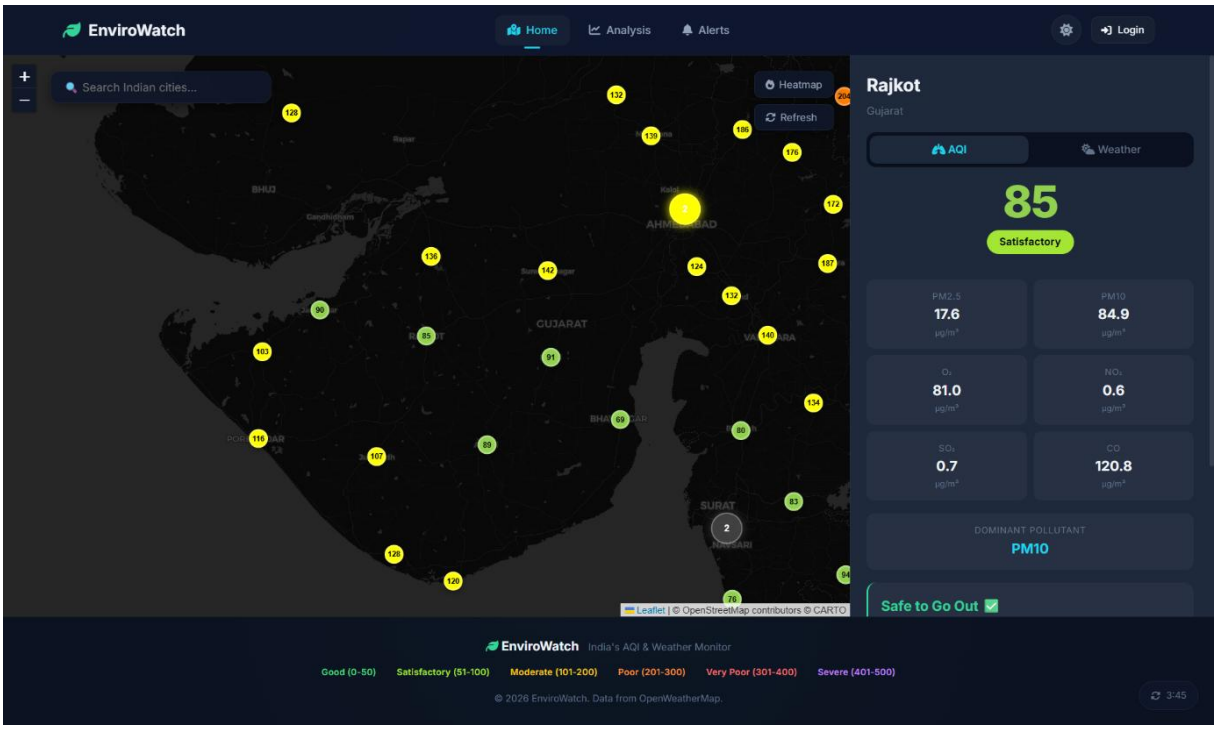
[8] GitHub, "Authorizing OAuth Apps," GitHub Docs, [Online]. Available: https://docs.github.com/en/apps/oauth-apps.

Project Outcome Documents

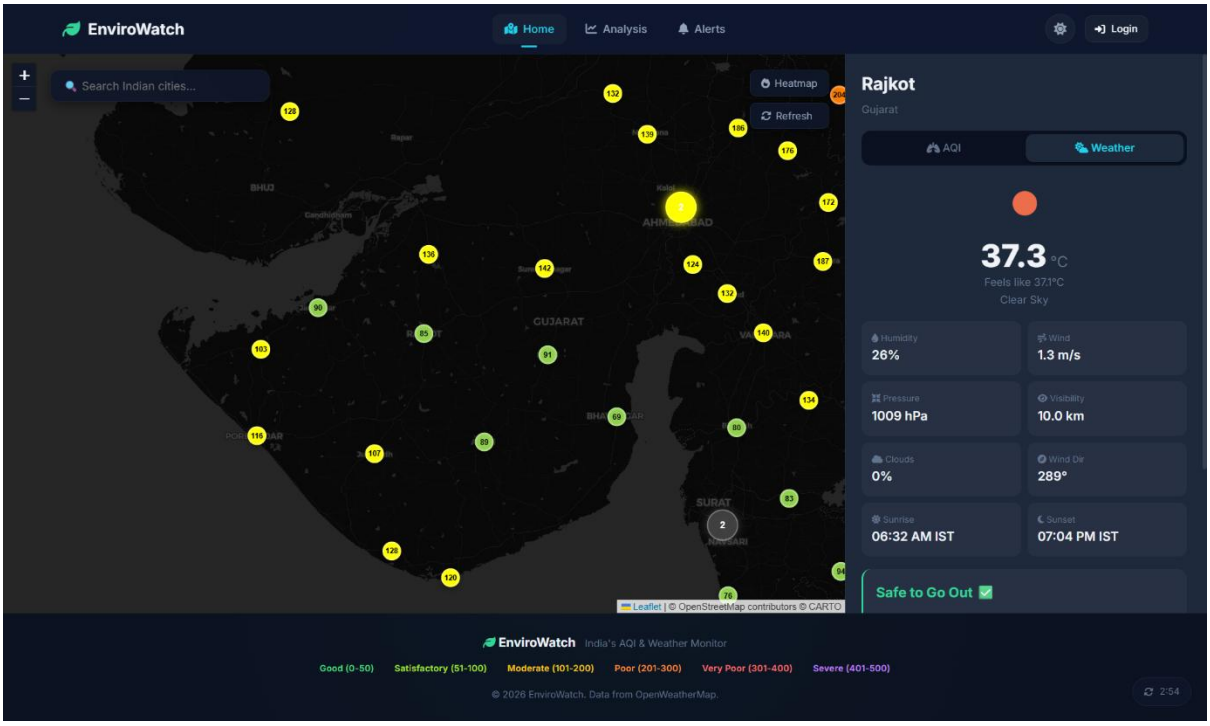
Screen 1: Home Page — Map with City



Screen 2: City with AQI



Screen 3: City with Weather



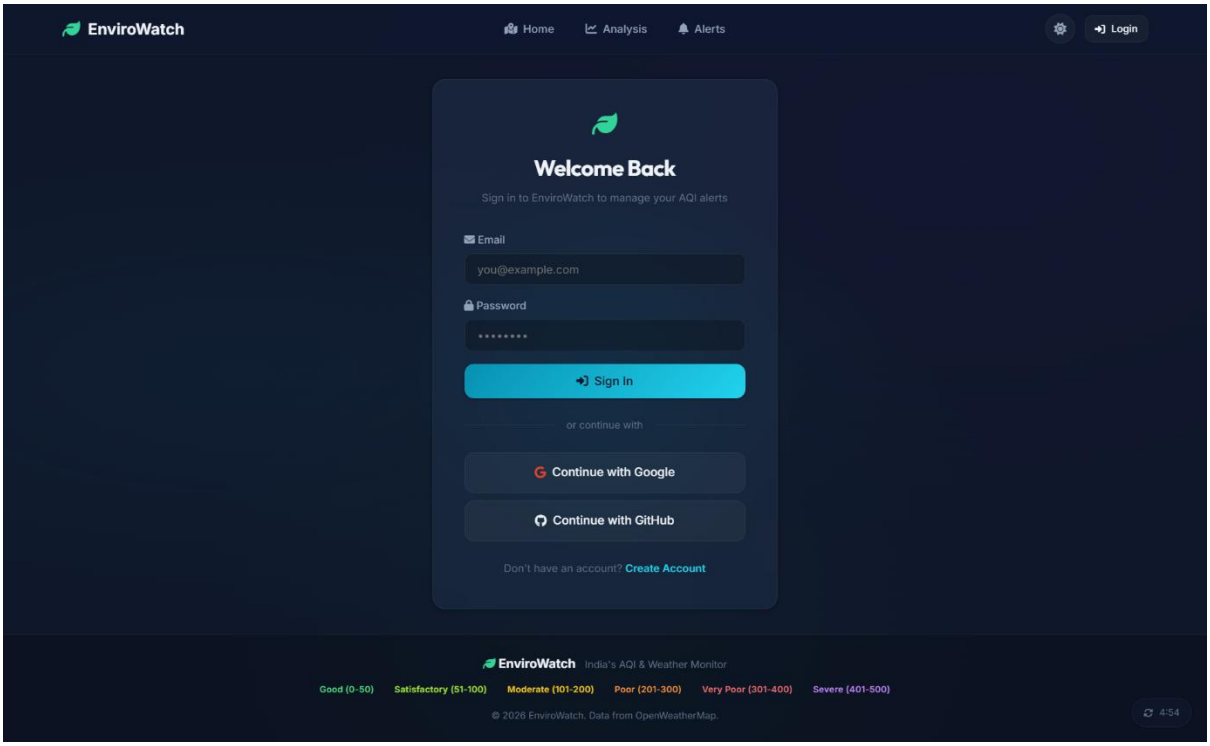
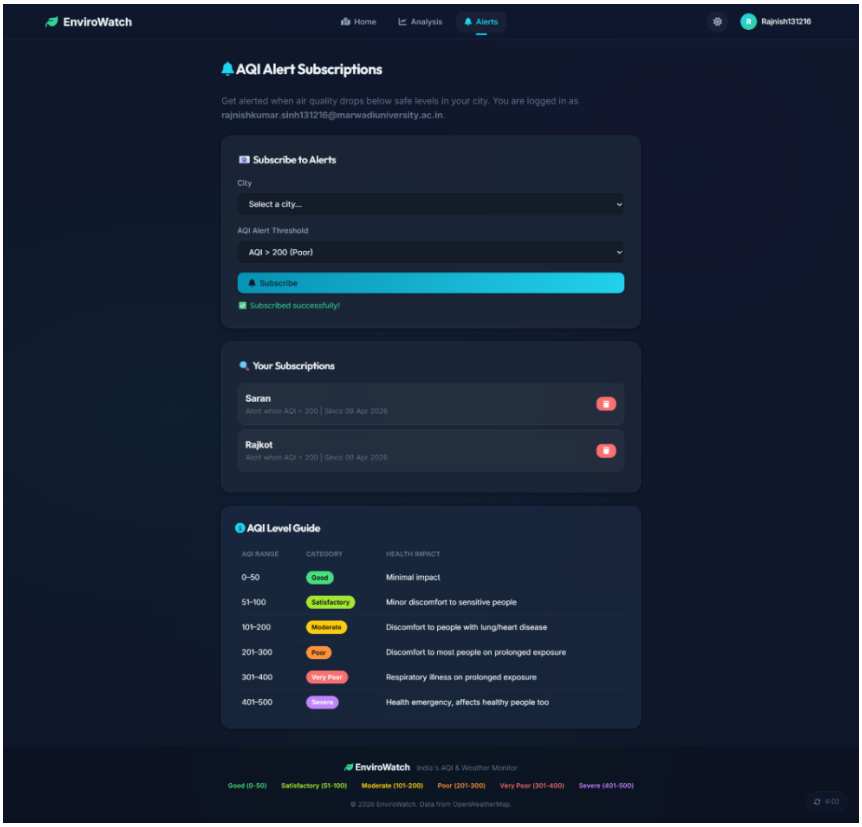
Screen 4: India – Air Quality & Weather Analysis

Screen 5: City - Air Quality & Weather Analysis (Last 7 Days)

Screen 6: City - Air Quality & Weather Analysis (Last 30 Days)

Screen 7: City Comparison (Upto 10 cities)

Screen 8: Sign In / Login



Screen 9: AQI Alert Subscription